Consensus Algorithms

Consensus is the process that allows all processes in a network to agree on some specific value in the presence of faults.

Distributed systems are classified into two main categories:

- Message passing
- Shared memory.

In the context of blockchain, we are concerned with the message-passing type of distributed systems, where participants on the network communicate with each other via passing messages to each other.

The consensus problem can also be classified based on these two paradigms.

For example, Bitcoin is a public blockchain. It runs PoW, also called Nakamoto consensus. In contrast, many permissioned blockchains tend to run variants of traditional or classical distributed consensus.

Fault tolerance

A fundamental requirement in a consensus mechanism is fault tolerance in a network, and it should continue to work even in the presence of faults. This naturally means that there must be some limit to the number of faults a network can handle since no network can operate correctly if many of its nodes fail. Based on the fault-tolerance requirement, consensus algorithms are also called fault-tolerant algorithms, and there are two types of fault-tolerant algorithms:

Faulty node just stops — easy to detect.

Crash fault-tolerance (CFT), which covers only crash faults or, in other words, benign faults.

Byzantine fault-tolerance (BFT), which deals with the type of faults that are arbitrary and can even be malicious.

Faulty node may lie, send conflicting messages, or act maliciously — much harder.

In the case of CFT, at least $2f + 1$ number of nodes is required to achieve consensus.

In the case of BFT, at least $3f + 1$ number of nodes is required to achieve consensus.

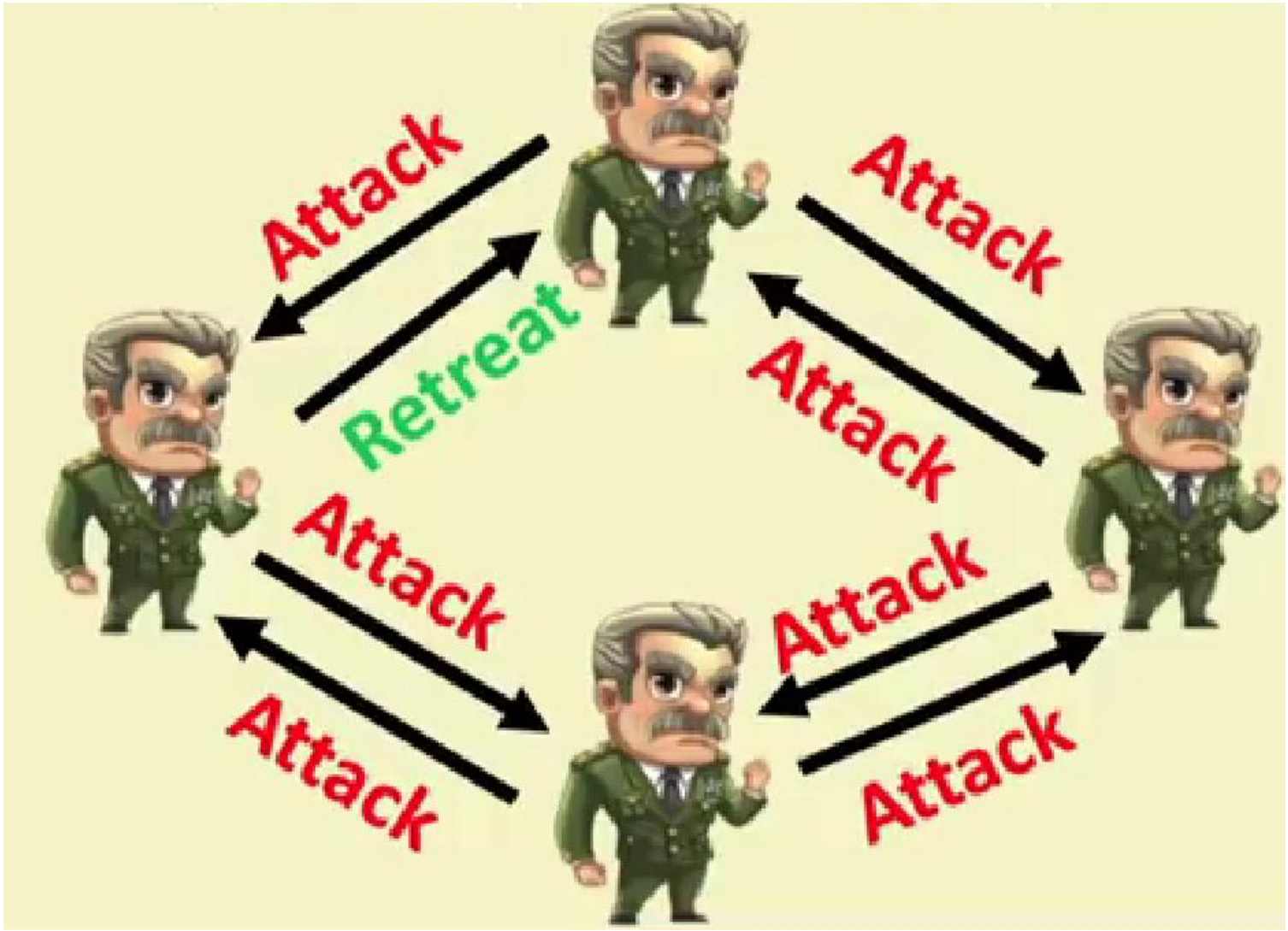
where f represents the number of failed nodes.

Fault Tolerance: The Byzantine Army Problem

Byzantine army are encamped around a city they plan to attack. The generals must decide whether to attack or retreat. They face two key challenges:

1.Coordination: All generals must agree on the same decision (attack or retreat). A lack of consensus could result in a disastrous defeat.

2.Traitors: Some generals may be traitors and send conflicting or false messages to disrupt the decision-making process.



Classification of Consensus Algorithms

The consensus algorithms can be classified into two broad categories:

Traditional: Voting-based consensus. Traditional voting-based consensus has been researched in distributed systems for many decades. Many fundamental results and a lot of ground-breaking work have already been produced in this space. Algorithms like Paxos and PBFT are prime examples of such types of algorithms. Traditional consensus can also be called fault-tolerant distributed consensus.

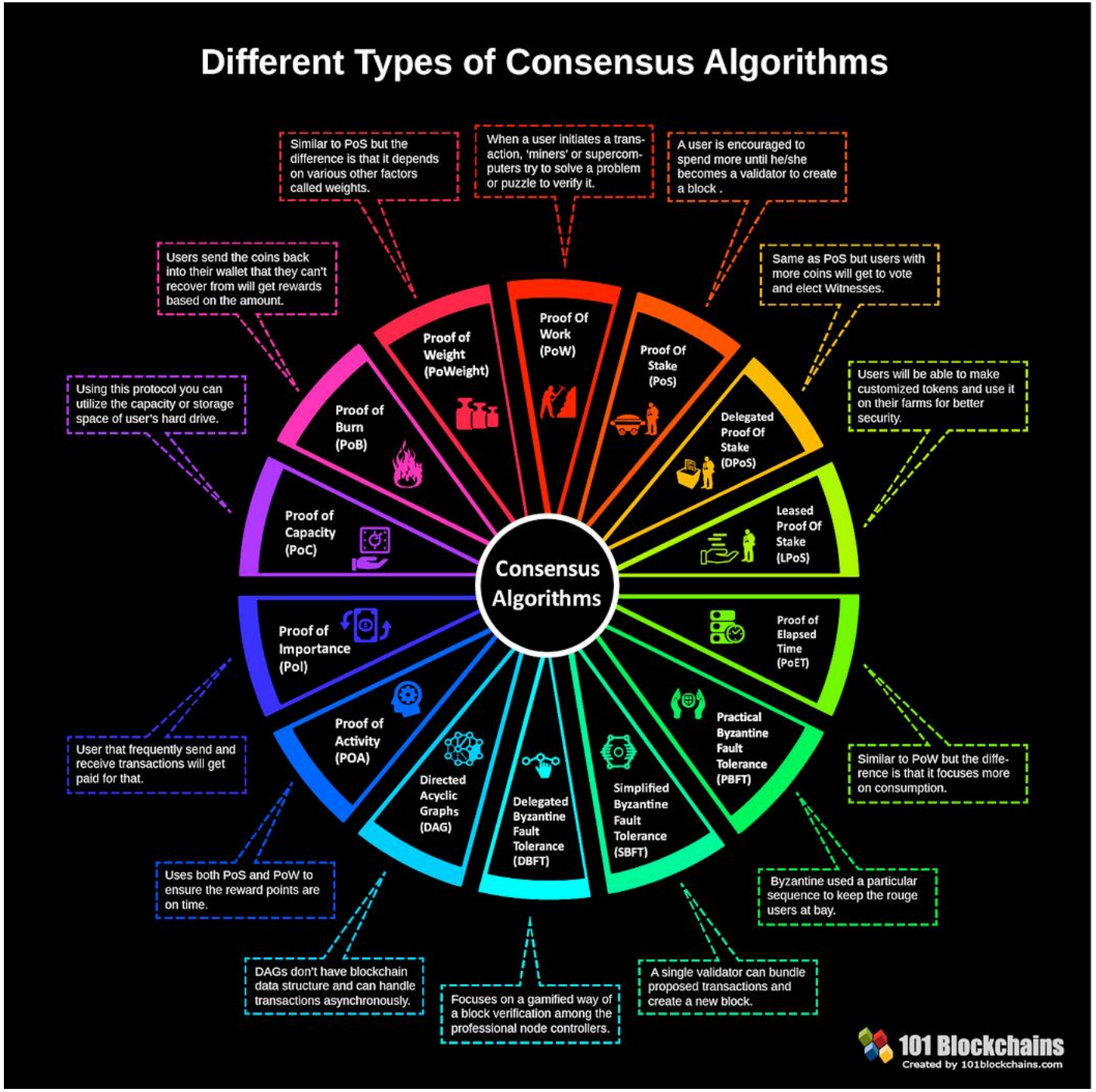
Lottery-based: Nakamoto and post-Nakamoto consensus. Lottery-based or Nakamoto-type consensus was first introduced with Bitcoin. This class can also be simply called blockchain consensus.

Requirements for Consensus Algorithms

The fundamental requirements of consensus algorithms boil down to Safety and Liveness conditions.

- The **Safety** requirement generally means that nothing bad happens, and is usually based on some safety requirements of the algorithms, such as agreement, validity, and integrity.
  - Agreement: The agreement property requires that no two processes decide on different values.
  - Validity: Validity states that if a process has decided on a value, that value must have been proposed by a process. In other words, the decided value is always proposed by an honest process and has not been created out of thin air.
  - Integrity: A process must decide only once.
- The **Liveness** requirement generally means that something good eventually happens. It means that the protocol can make progress even if the network conditions are not ideal. It usually has one requirement:
  - Termination: This liveness property states that each honest node must eventually decide on a value.

Different Consensus Algorithms



CFT Algorithms

Crash fault tolerant is a name given the algorithms that solve the problem of nodes crashing simply. So, we have a decentralized system where nodes can halt or disconnect from the network and yet we maintain the same state of truth on the system.

Paxos: CFT Algorithm

- The most fundamental distributed consensus algorithm, allowing consensus over a value under unreliable communications.
- Paxos is used to build a reliable system that works correctly, even in the presence of faults.
- There are many other protocols that have since emerged from basic Paxos, such as Multi-Paxos, Fast Paxos, and Cheap Paxos.
- Paxos works under an asynchronous network model and supports the handling of only benign failures, that is, it does not support Byzantine fault-tolerant protocol.
- However, later, a variant of Paxos was developed that provides BFT.

Paxos: CFT Algorithm

- Paxos makes use of $2f + 1$ processes to ensure fault tolerance in a network where processes can crash fault, that is, experience benign failures.
- Benign failure means either the loss of a message or a process stops.
- Paxos can tolerate one crash failure in a three-node network.
- Paxos is a two-phase protocol. The first phase is called the prepare phase, and the next phase is called the accept phase.
- Paxos has a proposer and acceptors as participants, where the proposer is the replicas or nodes that propose the values, and acceptors are the nodes that accept the values.
- The Paxos protocol assumes an asynchronous message-passing network with less than 50% of crash faults.
- The critical properties of the Paxos consensus algorithm are safety and liveness.

Paxos: CFT Algorithm

- Processes can assume different roles, which are listed as follows:
  - Proposers are elected leader(s) that can propose a new value to be decided.
  - Acceptors participate in the protocol as a means to provide a majority decision.
  - Learners are nodes that just observe the decision process and value.
- A single process in a Paxos network can assume all three roles.
- The key idea behind Paxos is that the proposer node proposes a value, which is considered final only if a majority of the acceptor nodes accept it. The learner nodes also learn this final decision.

Paxos: How does it work?

Paxos protocol works step by step as given below:

1. The proposer proposes a value by broadcasting a message, , to all acceptors.
2. Acceptors respond with an acknowledgment message if proposal n is the highest that the acceptor has responded to so far. The acknowledgment message consists of three variables, where n is the proposal number, v is the proposal value of the highest numbered proposal the acceptor has accepted so far, and s is the sequence number of the highest proposal accepted by the acceptor so far. This is where acceptors agree to commit the proposed value. The proposer now waits to receive acknowledgment messages from the majority of the acceptors indicating the chosen value.
3. If the majority is received, the proposer sends out the “accept” message to the acceptors.
4. If the majority of the acceptors accept the proposed value (now the “accept” message), then it is decided: that is, an agreement is achieved.

Paxos: How does it work?

5. Finally, in the learning phase, acceptors broadcast the "accepted" message to the proposer. This phase is necessary to disseminate which proposal has been finally accepted. The proposer then informs all other learners of the decided value. Alternatively, learners can learn the decided value via a message that contains the accepted value (decision value) multicast by acceptors.

❑ Paxos ensures its safety and liveness guarantees. Paxos achieves all these properties efficiently.

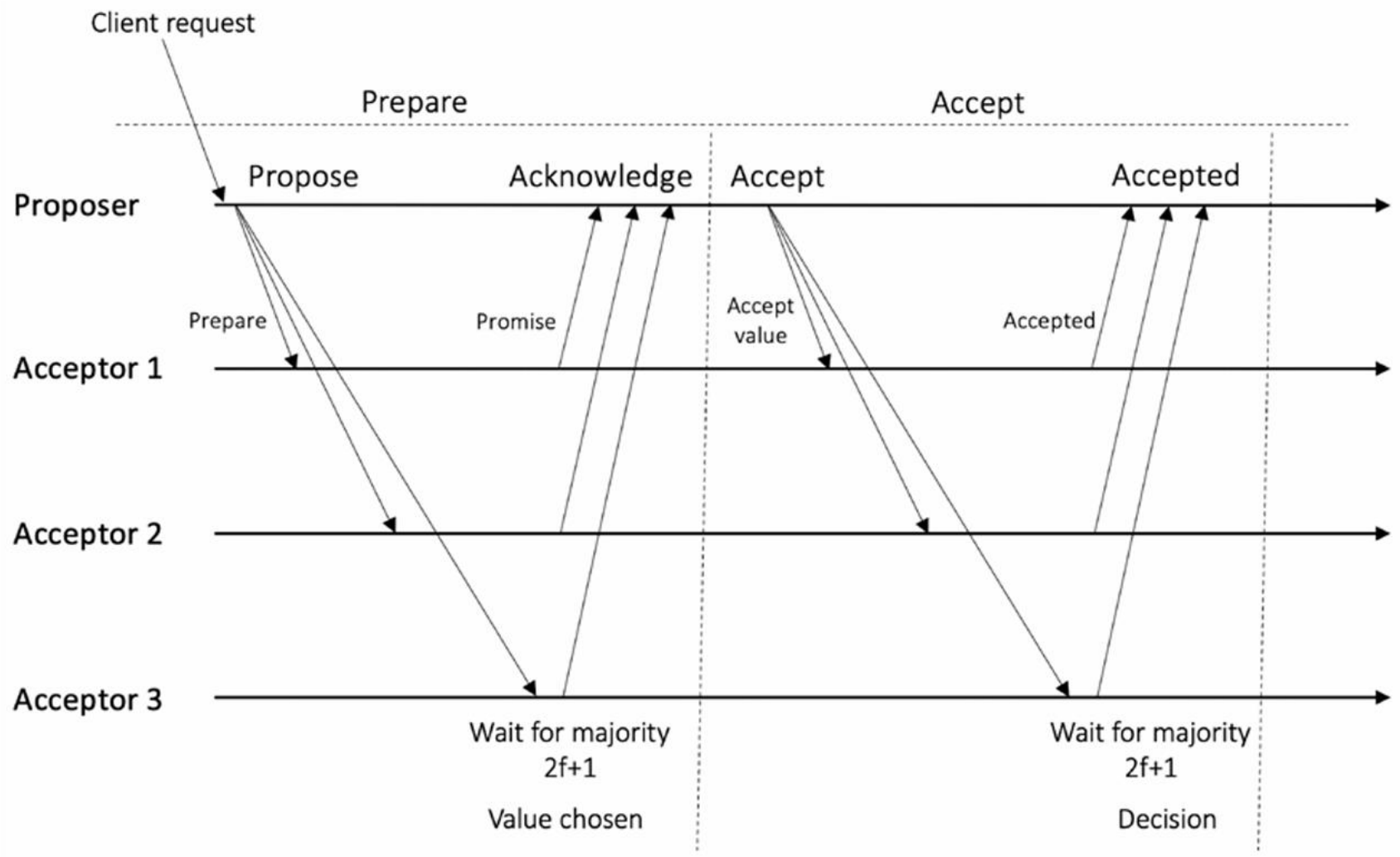
❖ Agreement is ensured by enforcing that only one proposal can win votes from a majority of the acceptors.

❖ Validity is ensured by enforcing that only genuine proposals are decided. In other words, no value is committed unless it is proposed in the proposal message first.

❖ Liveness, or termination, is guaranteed by ensuring that at some point during the protocol execution, eventually, there is a period during which there is only one fault-free proposer.

Paxos: How does it work?

Figure: How Paxos works



Replication in Distributed Systems

Replication is a standard approach to improve the fault tolerance and availability of a network. Replication results in a synchronized copy of data across all nodes in a network. This means that even if some of the nodes become faulty, the overall system/network remains available due to the data being available on multiple nodes.

There are two main types of replication techniques:

- Active replication, which is a type where each replica becomes a copy of the original state machine replica.
- Passive replication, which is a type where there is only a single copy of the state machine in the system kept by the primary node, and the rest of the nodes/replicas only maintain the state.

Active Replication in Distributed Systems

Active replication is a fault-tolerance approach in distributed systems where all replicas (servers) process client requests independently and simultaneously. All replicas maintain the same state and perform the same operations, ensuring consistency.

Example: Online Voting System

Consider an online voting system where users vote for candidates, and the results must remain consistent even if some servers fail.

Setup:

1. Replicas (Active Servers):

1. Multiple servers (e.g., Server A, Server B, Server C) are running the same application.
2. All servers receive and process the same client requests.

Active Replication in Distributed Systems

2. Request Distribution:

1. A client's vote (e.g., "Vote for Candidate X") is sent to all replicas.
2. Each replica processes the vote independently and updates its local state.

3. Workflow:

1. A user submits a vote for Candidate X.
2. The request is broadcast to all replicas using a reliable multicast protocol (e.g., Paxos or Raft).
3. Each replica:
    1. Processes the request independently.
    2. Updates its local vote count for Candidate X.
4. After processing, the replicas maintain a consistent state (e.g., Candidate X has 100 votes across all servers).
5. If a client queries any replica for the current vote count, it receives the same result regardless of which server responds.

Active Replication in Distributed Systems

Fault Tolerance:

If one or more replicas fail, the system continues to operate using the remaining replicas.

New replicas can be added and synchronized with the existing state.

Advantages of Active Replication:

1. **High Availability:** No single point of failure, as all replicas are active and can respond to client requests.
2. **Consistency:** Consistent state is maintained across all replicas due to deterministic processing of identical requests.

Active Replication in Distributed Systems

Limitations:

1. **Increased Overhead:** Processing the same request on all replicas leads to redundant computation.
2. **Synchronization Complexity:** Requires precise ordering of requests (e.g., using consensus algorithms) to ensure all replicas process them in the same sequence.

Active replication is commonly used in **mission-critical systems**, such as flight control systems, financial transaction systems, and distributed databases, where availability and consistency are paramount.

Passive Replication in Distributed Systems

Passive replication, is a fault-tolerance technique in distributed systems where one server (the primary) handles all client requests and updates. The backups remain passive and synchronize their state with the primary to take over in case of failure.

Example: Bank Account Management System

Imagine a distributed banking system where clients can deposit or withdraw money from their accounts. To ensure fault tolerance, the system uses passive replication.

Passive Replication in Distributed Systems

Setup:

1. Primary Server (Active):

1. Handles all client requests (e.g., deposit, withdrawal).
2. Updates the account balance.

2. Backup Servers (Passive):

1. Do not handle client requests directly.
2. Receive state updates from the primary server.
3. Synchronize periodically or after every transaction.

Passive Replication in Distributed Systems

3. Workflow:

1. A client sends a deposit request of $100 to the primary server.
2. The primary server processes the request:
    1. Updates the account balance (e.g., from $500 to $600).
    2. Sends the updated balance and transaction log to the backup servers.
3. Backup servers store the updated balance and log but do not process the request directly.
4. If the primary server fails:
    1. One of the backup servers is promoted to become the new primary.
    2. It continues processing client requests using the synchronized state.

Replication in Distributed Systems

❑ **This approach ensures:**

➢ Fault tolerance: The system can continue functioning even if the primary fails.

➢ Consistency: Backups always maintain a consistent state with the primary.

❑ **Limitations:**

➢ Latency: Synchronizing backups can introduce delays.

➢ Single point of failure during updates: If the primary fails before backups are updated, some data may be lost.

Passive replication is widely used in systems like databases, cloud services, and financial applications where strong consistency is crucial.

Timing assumptions in Consensus Algorithms Design

There are also some timing assumptions that are made when designing consensus algorithms:

Synchrony: There is a known upper bound on communication and processor delays. Synchronous algorithms are designed to be run on synchronous networks. In a synchronous system, a message sent by a processor to another is received by the receiver in the same communication round as it is sent.

Asynchrony: In asynchronous systems, there is no upper bound on communication and processor delays. Asynchronous algorithms are designed to run on asynchronous networks without any timing assumptions. These systems are characterized by the unpredictability of message transfer (communication) delays and processing delays.

Partial synchrony: There is an upper bound on the communication and processor delays, However, this upper bound is not known to the processors. An eventually synchronous system is a type of partial synchrony, which means that the system becomes synchronous after an instance of time.

BFT Algorithms

A **Byzantine fault** refers to a situation where components of the system (e.g., nodes, processes) can fail and provide conflicting or malicious information to other components. These faults are the most challenging because they involve arbitrary or malicious behavior rather than simple crashes or benign failures.

- Practical Byzantine Fault Tolerance

Practical Byzantine Fault Tolerance (PBFT) is a protocol developed to provide consensus in the presence of Byzantine faults. This algorithm demonstrated for the first time that PBFT is possible.

- Istanbul Byzantine Fault Tolerance

The Istanbul Byzantine Fault Tolerance (IBFT) algorithm is a specific implementation of the Byzantine Fault Tolerance (BFT) consensus mechanism. It is commonly used in blockchain systems, particularly in enterprise or private blockchain platforms like Ethereum-based Quorum and Polygon (formerly Matic Network).

Practical BFT Algorithms

PBFT comprises three subprotocols:

Normal operation subprotocol refers to a scheme that is executed when everything is running normally, and no errors are in the system.

View change is a subprotocol that runs when a faulty leader node is detected in the system.

Checkpointing is another subprotocol, which is used to discard old data from the system.

The protocol runs in rounds where, in each round, an elected leader or primary node handles the communication with the client. In each round, the protocol progresses through three phases or steps. These phases are pre-prepare, prepare, and commit.

In order to tolerate Byzantine faults, the minimum number of nodes required is $n = 3f + 1$, where n is the number of nodes and f is the number of faulty nodes. PBFT ensures BFT as long as the number of nodes in a system stays $n >= 3f + 1$.

Practical BFT Algorithms

Pre-prepare:

The main purpose of this phase is to assign a unique sequence number to the request. This is the first phase in the protocol, where the primary node receives (accepts) a request from the client. The primary node assigns a sequence number to the request. It then sends the pre-prepare message with the request to all backup replicas.

When the pre-prepare message is received by the backup replicas, it checks a number of things to ensure the validity of the message:

1. First, whether the digital signature is valid.
2. After this, whether the current view number is valid.
3. Then, whether the sequence number of the operation's request message is valid.
4. Finally, whether the digest / hash of the operation's request message is valid.

If all of these elements are valid, then the backup replica accepts the message. After accepting the message, it updates its local state and progresses toward the prepare phase.

Practical BFT Algorithms

Prepare:

- This phase ensures that honest replicas/nodes in the network agree on the total order of requests within a view.
- The pre-prepare and prepare phases together provide the total order to the messages.
- A prepare message is sent by each backup to all other replicas in the system.
- Each backup waits for at least 2f+1 prepare messages to be received from other replicas. They also check whether the prepare message contains the same view number, sequence number, and message digest values.
- If all these checks pass, then the replica updates its local state and progresses toward the commit phase.

Practical BFT Algorithms

Commit:

- This phase ensures that honest replicas/nodes in the network agree on the total order of requests across views.
- In this phase, each replica sends a commit message to all other replicas in the network and wait for 2f+1 commit messages to arrive from other replicas.
- The replicas also check the view number, sequence number, and message digest values. If they are valid for 2f+1 commit messages received from other replicas, then the replica executes the request, produces a result, and finally, updates its state to reflect a commit.
- If there are already some messages queued up, the replica will execute those requests first before processing the latest sequence numbers.
- Finally, the replica sends the result to the client in a reply message. The client accepts the result only after receiving 2f+1 reply messages containing the same result.

Visualization of Practical BFT Algorithms

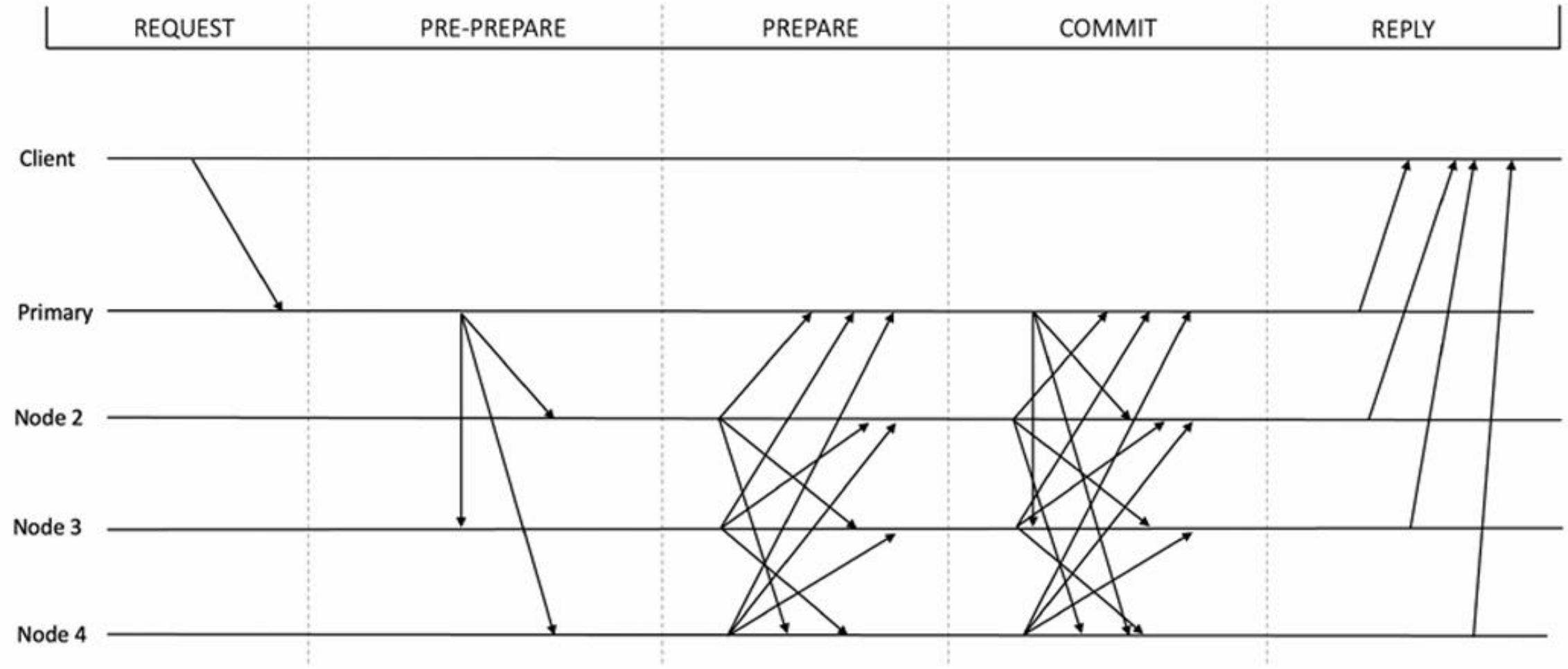


Figure: Practical BFT protocol

Nakamoto Consensus

- Nakamoto consensus is also known as Proof of Work(PoW).
- It was first introduced with Bitcoin in 2009.
- It has stood the test of time and is the longest-running blockchain network.
- The PoW mechanism is not a consensus algorithm but a Sybil attack defense mechanism.
- The consensus is actually achieved by applying the fork choice rule to choose the longest/heaviest chain to finalize the canonical chain.
- We can say that PoW is a consensus facilitation algorithm that facilitates consensus and mitigates Sybil attacks, thus maintaining the blockchain's consistency and network's security.

Nakamoto Consensus

What is Sybil attack?

A Sybil attack is a type of attack that aims to gain a majority influence on the network to control the network. Once a network is under the control of an adversary, any malicious activity could occur. A Sybil attack is usually conducted by a node generating and using multiple identities on the network. If there are enough multiple identities held by an entity, then that entity can influence the network by skewing majority-based network decisions. The majority in this case is held by the adversary.

Background of PoW:

- The key idea behind PoW as a solution to the Byzantine generals problem is that all honest generals (miners in the Bitcoin world) achieve agreement on the same state (decision value). As long as honest participants control the majority of the network, PoW solves the Byzantine generals problem. Note that this is a probabilistic solution and not deterministic.

Nakamoto Consensus

How PoW works?

PoW makes use of hash puzzles.

A node that proposes a block has to find a nonce such that H (nonce || previous hash || Tx || ... || Tx) < Threshold value. This threshold value or the target value is based on the mining difficulty of the network, which is set by adding or reducing the number of zeroes in front of the target hash value. More zeroes mean more difficulty and a smaller number of zeroes means less difficult to solve. If there are more miners in the network, then the mining difficulty increases, otherwise, it reduces as the number of miners reduces.

The process can be summarized as follows:

1. New transactions are broadcast to the network.
2. Each node collects the transactions into a candidate block.
3. Miners propose new blocks.

Consensus Algorithms

How PoW works?

4. Miners concatenate and hash with the header of the previous block.
5. The resultant hash is checked against the target value, that is, the network difficulty target value.
6. If the resultant hash is less than the threshold value (the target value), then PoW is solved, otherwise, the nonce is incremented and the node tries again. This process continues until a resultant hash is found that is less than the threshold value.

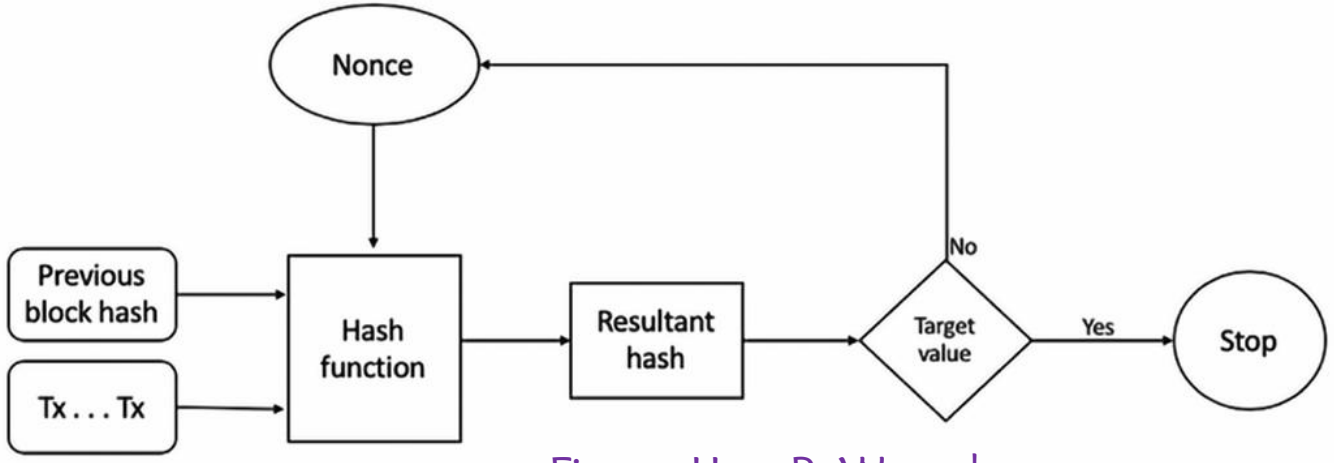


Figure: How PoW works.

Nakamoto Consensus

How Nakamoto Consensus meets the traditional consensus?

The Nakamoto consensus was the first of its kind. It solved the consensus problem, which had been traditionally solved using pre-Bitcoin protocols like PBFT. However, we can map the properties of PoW consensus to traditional Byzantine consensus.

In traditional consensus algorithms, we have agreement, validity, and liveness properties, which can be mapped to Nakamoto-specific properties of the common prefix, chain quality, and chain growth properties respectively:

The common prefix property means that the blockchain hosted by honest nodes will share the same large common prefix. If that is not the case, then the agreement property of the protocol cannot be guaranteed, meaning that the processors will not be able to decide and agree on the same value.

The chain quality property means that the blockchain contains a certain required level of correct blocks created by honest nodes (miners). If chain quality is compromised, then the validity property of the protocol cannot be guaranteed. This means that there is a possibility that a value will be decided that is not proposed by a correct process, resulting in safety violations.

The chain growth property simply means that new correct blocks are continuously added to the blockchain. If chain growth is impacted, then the liveness property of the protocol cannot be guaranteed. This means that the system can deadlock or fail to decide on a value.